

12. ВНЕШНИЕ ФУНКЦИОНАЛЬНЫЕ ИНТЕРФЕЙСЫ (FOREIGN FUNCTION INTERFACES)

Не весь код написан на Rust. Знаю, это шокирует. Время от времени вам придётся взаимодействовать с кодом, написанным на других языках, — либо вызывая такой код из Rust, либо позволяя этому коду вызывать ваш Rust-код. Добиться этого можно с помощью интерфейсов внешних функций (foreign function interfaces, FFI).

В этой главе мы сначала рассмотрим основной механизм, который Rust предоставляет для FFI: ключевое слово `extern`. Мы увидим, как использовать `extern` и для того, чтобы открывать Rust-функции и статические переменные другим языкам, и для того, чтобы давать Rust доступ к функциям и статическим переменным, предоставленным извне Rust-пузыря. Затем мы шаг за шагом разберём, как согласовывать типы Rust с типами, определёнными в других языках, и исследуем некоторые тонкости передачи данных через границу FFI. Наконец, мы поговорим о некоторых инструментах, которые вам, скорее всего, понадобятся, если вы будете заниматься FFI в сколько-нибудь нетривиальных масштабах.

ПРИМЕЧАНИЕ Хотя я часто говорю об FFI как о пересечении границы между одним языком и другим, FFI может также целиком происходить внутри Rust-страны. Если одна Rust-программа разделяет память с другой Rust-программой, но обе не скомпилированы вместе — скажем, если вы используете динамически подключаемую библиотеку в своей Rust-программе, которая сама написана на Rust, но у вас есть лишь C-совместимый файл `.so`, — возникают те же самые сложности.

Пересечение границ с помощью `extern` (Crossing Boundaries with `extern`)

FFI в конечном счёте полностью посвящён доступу к байтам, которые возникли где-то вне Rust-кода вашего приложения. Для этого Rust

предоставляет два основных строительных блока: символы (symbols) — имена, присвоенные конкретным адресам в заданном сегменте вашего двоичного файла, которые позволяют вам разделять память (будь то данные или код) между внешним источником и вашим Rust-кодом, — и соглашения о вызове (calling conventions), которые задают общее понимание того, как вызывать функции, хранящиеся в такой разделяемой памяти. Мы рассмотрим каждый из них по очереди.

Символы (Symbols)

Любой двоичный артефакт, который компилятор производит из вашего кода, наполнен символами — у каждой функции или статической переменной, которую вы определяете, есть символ, указывающий на её расположение в скомпилированном двоичном файле. У обобщённых функций может быть даже несколько символов — по одному на каждую мономорфизацию функции, которую генерирует компилятор!

Обычно вам не приходится думать о символах — они используются компилятором внутренне для передачи окончательного адреса функции или статической переменной по вашему двоичному файлу. Именно так компилятор знает, на какое место в памяти должен быть нацелен каждый вызов функции, когда он генерирует окончательный машинный код, или откуда читать, если ваш код обращается к статической переменной. Поскольку вы обычно не ссылаетесь на символы напрямую в своём коде, компилятор по умолчанию выбирает для них полуслучайные имена — у вас может быть две функции с именем `foo` в разных частях кода, но компилятор сгенерирует из них различные символы, чтобы не возникало путаницы.

Однако использование случайных имён для символов не сработает, когда вы хотите вызвать функцию или обратиться к статической переменной, которая скомпилирована в то же время, например к коду, написанному на другом языке и потому скомпилированному другим компилятором. Вы не сможете сообщить Rust о статической переменной, определённой в C, если символ для этой переменной имеет полуслучайное имя, которое постоянно меняется. И наоборот, вы не сможете рассказать FFI-интерфейсу Python о Rust-функции, если не сумеете создать для неё стабильное имя.

Чтобы использовать символ с внешним происхождением, нам также нужен способ сообщить Rust о переменной или функции таким образом, чтобы

компилятор искал этот же символ, определённый где-то ещё, а не определял свой собственный (как происходит этот поиск, мы поговорим позже). Иначе мы получили бы два идентичных символа для этой функции или статической переменной, и никакого разделения не происходило бы. По сути, вероятнее всего, компиляция вообще завершилась бы неудачей, поскольку любой код, ссылающийся на этот символ, не знал бы, какое определение (то есть какой адрес) использовать для него!

ПРИМЕЧАНИЕ Краткое замечание о терминологии: символ может быть *объявлен* много раз, но *определён* только один раз. Каждое объявление символа будет привязано к тому же единственному определению этого символа во время компоновки. Если для объявления не найдено определения или если определений несколько, компоновщик пожалуется.

Отступление о компиляции и компоновке (An Aside on Compilation and Linking)

Время для экспресс-курса по компиляторам! Примерное понимание сложного процесса превращения кода в исполняемый двоичный файл поможет вам лучше понять FFI.

Компилятор — это не одна монолитная программа, а (как правило) совокупность более мелких программ, каждая из которых выполняет отдельные задачи и запускается одна за другой. На высоком уровне есть три различные фазы компиляции — *компиляция* (compilation), *генерация кода* (code generation) и *компоновка* (linking), — за которые отвечают три разных компонента.

Первую фазу выполняет то, что большинство людей склонны представлять себе как «компилятор»; она занимается проверкой типов, проверкой заимствований, мономорфизацией и другими возможностями, которые мы ассоциируем с конкретным языком программирования. Эта фаза не генерирует машинный код, а скорее низкоуровневое представление кода, использующее обильно аннотированные абстрактные машинные операции. Это низкоуровневое представление затем передаётся инструменту генерации кода, который и производит машинный код, способный действительно выполняться на конкретном CPU.

Эти две операции, взятые вместе, не обязаны выполняться за один большой проход по всей кодовой базе сразу. Вместо этого кодовую базу можно нарезать на более мелкие фрагменты, которые затем прогоняются через компиляцию параллельно. Например, Rust обычно компилирует разные крейты независимо и параллельно, пока между ними нет зависимости. Он также может вызывать инструмент генерации кода для независимых крейтов по отдельности, чтобы обрабатывать их параллельно. Rust часто может даже компилировать несколько мелких фрагментов одного крейта по отдельности!

После того как машинный код для каждого фрагмента приложения сгенерирован, эти фрагменты можно соединить вместе. Это делается на фазе компоновки, что неудивительно, компоновщиком (linker). Основная задача компоновщика — взять все двоичные артефакты, называемые *объектными файлами* (object files), произведённые генерацией кода, сшить их вместе в один файл, а затем заменить каждую ссылку на символ окончательным адресом этого символа в памяти. Именно так вы можете определить функцию в одном крейте и вызвать её из другого, но при этом скомпилировать эти два крейта по отдельности!

Компоновщик — это то, что делает FFI возможным. Ему всё равно, как был сконструирован каждый из входных объектных файлов; он просто старательно компонуется все объектные файлы вместе, а затем разрешает любые разделяемые символы. Один объектный файл мог изначально быть Rust-кодом, другой — кодом на C, а ещё один — двоичным блоком, загруженным из интернета; пока все они используют одни и те же имена символов, компоновщик позаботится о том, чтобы результирующий машинный код использовал правильные перекрёстные адреса для любых разделяемых символов.

Символ может быть скомпонован либо *статически* (statically), либо *динамически* (dynamically). Статическая компоновка проще всего, поскольку каждая ссылка на символ просто заменяется адресом определения этого символа во время компиляции. Динамическая компоновка, с другой стороны, привязывает каждую ссылку на символ к небольшому фрагменту сгенерированного кода, который пытается найти определение символа во время *выполнения* программы. Мы поговорим об этих режимах компоновки чуть позже. Rust обычно использует статическую компоновку для Rust-кода и динамическую компоновку для FFI.

Использование `extern` (Using `extern`)

Ключевое слово `extern` — это механизм, который позволяет нам объявить, что символ принадлежит внешнему интерфейсу. Его можно использовать и для того, чтобы объявить и определить символ для экспорта через FFI, и для того, чтобы объявить символ, определённый где-то ещё. В листинге 12-1 мы определяем статическую переменную с именем `RS_DEBUG` на Rust, которую делаем доступной другим языкам через FFI. Мы также объявляем статическую переменную `FOREIGN_DEBUG`, чьё определение не указано и которое будет разрешено во время компоновки.

```
#[no_mangle]
pub static RS_DEBUG: bool = true;

extern {
    static FOREIGN_DEBUG: bool;
}
```

Листинг 12-1: Открытие Rust-статической переменной и доступ к переменной, объявленной в другом месте, через FFI

Атрибут `#[no_mangle]` гарантирует, что `RS_DEBUG` сохранит это имя во время компиляции, а не получит другое имя символа, присвоенное компилятором, чтобы, например, отличить её от другой (не-extern) статической переменной `RS_DEBUG` в программе. Переменная также объявлена как `pub`, поскольку она является частью публичного API крейта, хотя эта аннотация и не строго необходима для элементов, помеченных `#[no_mangle]`.

Блок `extern`, окружающий статическую переменную `FOREIGN_DEBUG`, обозначает, что это объявление ссылается на местоположение, о котором Rust узнает во время компоновки, исходя из того, где находится определение того же символа. Поскольку она определена в другом месте, мы не задаём ей начальное значение, а только тип, который должен совпадать с типом, использованным в месте определения. Поскольку Rust ничего не знает о коде, определяющем эту статическую переменную, и потому не может проверить, что вы объявили правильный тип для символа, к `FOREIGN_DEBUG` можно обращаться только внутри блока `unsafe`.

ПРИМЕЧАНИЕ Статические переменные в Rust по умолчанию не изменяемы, независимо от того, находятся ли они в блоке `extern`. Эти переменные всегда доступны из любого потока, так что изменяемый

доступ создавал бы риск гонки данных. Вы можете объявить `static as mut`, но тогда доступ к ней становится небезопасным (`unsafe`).

Процедура объявления FFI-функций очень похожа. В листинге 12-2 мы делаем `hello_rust` доступной для не-Rust-кода и подтягиваем внешнюю функцию `hello_foreign`.

```
#[no_mangle]
pub extern fn hello_rust(i: i32) { ... }

extern {
    fn hello_foreign(i: i32);
}
```

Листинг 12-2: Открытие Rust-функции и доступ к функции, определённой в другом месте, через FFI

Строительные блоки здесь точно такие же, как в листинге 12-1, за исключением того, что Rust-функция объявлена с помощью `extern fn`, что мы исследуем в следующем разделе.

Если есть несколько определений данного `extern`-символа, такого как `FOREIGN_DEBUG` или `hello_foreign`, вы можете явно указать, с какой библиотекой должен компоноваться символ, с помощью атрибута `#[link]`. Если вы этого не сделаете, компоновщик выдаст ошибку, сообщив, что нашёл несколько определений для рассматриваемого символа. Например, если вы предварите `extern`-блок атрибутом `#[link(name = "crypto")]`, вы говорите компоновщику разрешать любые символы (будь то статические переменные или функции) относительно скомпонованной библиотеки с именем «crypto». Вы также можете переименовать внешнюю статическую переменную или функцию в своём Rust-коде, аннотировав её объявление атрибутом `#[link_name = "<actual_symbol_name>"]`, и тогда элемент будет компоноваться с тем именем, какое вы пожелаете. Аналогично, вы можете переименовать Rust-элемент для экспорта с помощью `#[export_name = "<export_symbol_name>"]`.

Виды компоновки (Link Kinds)

`#[link]` также принимает аргумент `kind`, который определяет, как должны компоноваться элементы в блоке. По умолчанию этот аргумент равен `"dylib"`, что означает С-совместимую динамическую компоновку. Альтернативное

значение `kind` — `"static"`, которое указывает, что элементы в блоке должны компоноваться полностью во время компиляции (то есть статически). По сути это означает, что внешний код встраивается напрямую в двоичный файл, произведённый компилятором, и потому ему не нужно существовать во время выполнения. Есть и несколько других видов (`kinds`), но они гораздо менее распространены и выходят за рамки этой книги.

Между статической и динамической компоновкой есть несколько компромиссов, но главные соображения — это безопасность, размер двоичного файла и распространение. Во-первых, динамическая компоновка обычно более безопасна, поскольку она упрощает независимое обновление библиотек. Динамическая компоновка позволяет тому, кто развёртывает двоичный файл, содержащий ваш код, компоновать ваш код с библиотеками, не перекомпилируя его. Если, скажем, `libcrypto` получает обновление безопасности, пользователь может обновить `crypto`-библиотеку на хосте и перезапустить двоичный файл, и обновлённый код библиотеки будет использоваться автоматически. При статической компиляции код библиотеки жёстко зашит в двоичный файл, так что пользователю пришлось бы перекомпилировать ваш код относительно обновлённой версии библиотеки, чтобы получить обновление.

Динамическая компоновка также обычно производит меньшие двоичные файлы. Поскольку статическая компиляция включает любой скомпонованный код в окончательный двоичный вывод, как и любой код, который тот в свою очередь подтягивает, она производит более крупные двоичные файлы. При динамической компоновке каждый внешний элемент включает лишь небольшой фрагмент обёрточного кода, который загружает указанную библиотеку во время выполнения и затем перенаправляет доступ.

Пока что статическая компоновка может не казаться очень привлекательной, но у неё есть одно большое преимущество перед динамической компоновкой: простота распространения. При динамической компоновке любой, кто хочет запустить двоичный файл, включающий ваш код, должен *также* иметь любые библиотеки, с которыми компонуется ваш код. Мало того, он должен убедиться, что версия каждой такой библиотеки совместима с тем, что ожидает ваш код. Это может быть нормально для библиотек вроде `glibc` или `OpenSSL`, доступных в большинстве систем, но представляет проблему для более редких библиотек. Тогда пользователь должен знать, что ему нужно

установить эту библиотеку, и должен искать её, чтобы запустить ваш код! При статической компоновке код библиотеки встроен прямо в двоичный вывод, так что пользователю не нужно устанавливать её самому.

В конечном счёте не существует *правильного* выбора между статической и динамической компоновкой. Динамическая компоновка обычно является хорошим выбором по умолчанию, но статическая компиляция может быть лучшим вариантом для особенно ограниченных сред развёртывания или для очень мелких либо нишевых зависимостей-библиотек. Полагайтесь на собственное суждение!

Соглашения о вызове (Calling Conventions)

Символы определяют, *где* определена данная функция или переменная, но этого недостаточно, чтобы позволить вызовы функций через границы FFI. Чтобы вызвать внешнюю функцию на любом языке, компилятору также нужно знать её *соглашение о вызове* (calling convention), которое определяет, какой ассемблерный код использовать для вызова функции. Мы не будем здесь вдаваться в фактические технические детали каждого соглашения о вызове, но в качестве общего обзора: соглашение должно определять:

- как устроен кадр стека (stack frame) для вызова;
- как передаются аргументы (на стеке или в регистрах, по порядку или в обратном порядке);
- куда функции сказано вернуться, когда она завершится;
- как различные состояния CPU, такие как регистры, восстанавливаются у вызывающей стороны после завершения функции.

У Rust есть собственное уникальное соглашение о вызове, которое не стандартизировано и которое компилятору разрешено менять со временем. Это прекрасно работает, пока все определения функций и вызовы компилируются одним и тем же Rust-компилятором, но проблематично, если вы хотите взаимодействовать с внешним кодом, поскольку этот внешний код не знает Rust-соглашения о вызове.

Каждая Rust-функция неявно объявляется с `extern "Rust"`, если вы не объявите ничего другого. Использование `extern` само по себе, как в листинге 12-2, является сокращением для `extern "C"`, что означает «использовать стандартное C-соглашение о вызове». Сокращение существует потому, что C-соглашение о

вызове — это то, что вам нужно почти в каждом случае FFI.

ПРИМЕЧАНИЕ Раскрутка стека (unwinding) обычно работает только с обычными Rust-функциями. Если вы раскручиваете стек за конец Rust-функции, которая не является `extern "Rust"`, ваша программа аварийно завершится. Раскрутка через границу FFI во внешний код является неопределённым поведением. С RFC 2945 Rust получил новое объявление `extern`, `extern "C-unwind"`; оно разрешает раскрутку через границы FFI в определённых ситуациях, но если вы хотите его использовать, вам следует внимательно прочитать этот RFC.

Rust также поддерживает ряд других соглашений о вызове, которые вы задаёте в виде строки после ключевого слова `extern` (как в контексте `fn`, так и в контексте блока). Например, `extern "system"` говорит использовать соглашение о вызове стандартного библиотечного интерфейса операционной системы, которое на момент написания совпадает с `"C"` везде, кроме Win32, где используется соглашение о вызове `"stdcall"`. В целом вам редко понадобится явно задавать соглашение о вызове, если только вы не работаете с особенно платформозависимыми или сильно оптимизированными внешними интерфейсами, и просто `extern` (который есть `extern "C"`) будет нормально.

ПРИМЕЧАНИЕ Соглашение о вызове функции является частью её типа. То есть тип `extern "C" fn()` не то же самое, что `fn()` (или `extern "Rust" fn()`), который, в свою очередь, отличается от `extern "system" fn()`.

ПРОЧИЕ ДВОИЧНЫЕ АРТЕФАКТЫ (OTHER BINARY ARTIFACTS)

Обычно вы компилируете Rust-код только для того, чтобы запустить его тесты или собрать двоичный файл, который затем будете распространять или запускать. В отличие от многих других языков, вы обычно не компилируете Rust-библиотеку, чтобы распространять её среди других, — если вы выполните команду вроде `cargo publish`, она просто упакует исходный код вашего крейта и загрузит его на crates.io. Это в основном потому, что обобщённый код трудно распространять как-то иначе, чем в виде исходного кода. Поскольку компилятор мономорфизирует каждую обобщённую функцию относительно предоставленных аргументов-типов, а эти типы могут быть определены в крейте вызывающей стороны, компилятор должен иметь доступ к *обобщённой* форме функции, что означает отсутствие

оптимизированного машинного кода!

Технически Rust всё же компилирует двоичные библиотечные артефакты, называемые *rlib*, для каждой зависимости, которые в итоге объединяются. Эти *rlib* включают информацию, необходимую для разрешения обобщённых типов, но специфичны для конкретного использованного компилятора и в общем случае не могут быть осмысленно распространены.

Так что же делать, если вы хотите написать библиотеку на Rust, с которой затем захотите взаимодействовать из другого языка программирования? Решение — производить C-совместимые библиотечные файлы в виде динамически компокуемых библиотек (файлы `.so` в Linux, `.dylib` в macOS и `.dll` в Windows) и статически компокуемых библиотек (файлы `.a` в Linux/macOS и `.lib` в Windows). Эти файлы выглядят как файлы, произведённые C-кодом, так что их также можно использовать из C и других языков, которые умеют взаимодействовать с C.

Чтобы производить эти C-совместимые двоичные артефакты, вы задаёте поле `crate-type` в секции `[lib]` вашего файла `Cargo.toml`. Это поле принимает массив значений, который обычно был бы просто `"lib"`, чтобы указать стандартную Rust-библиотеку (*rlib*). Cargo применяет некоторые эвристики, которые задают это значение автоматически, если ваш крейт явно не является библиотекой (например, если это процедурный макрос), но рекомендуется задавать это значение явно, если вы производите что-либо, кроме старой доброй Rust-библиотеки.

Существует ряд различных типов крейтов, но релевантные здесь — это `"cdylib"` и `"staticlib"`, которые производят C-совместимые библиотечные файлы, динамически и статически компокуемые соответственно. Имейте в виду, что когда вы производите один из этих типов артефактов, доступны только публичные символы — то есть публичные и помеченные `#[no_mangle]` статические переменные и функции. Такие вещи, как типы и константы, будут недоступны, даже если они помечены `pub`, поскольку у них нет осмысленного представления в двоичном библиотечном файле.

Типы через границы языков (Types Across Language Boundaries)

При FFI компоновка типов критически важна; если один язык располагает в памяти некоторые разделяемые данные одним способом, а язык по другую сторону границы FFI ожидает, что они расположены иначе, то обе стороны будут интерпретировать данные как повреждённые. В этом разделе мы рассмотрим, как заставить типы согласовываться через FFI, а также другие аспекты типов, о которых нужно помнить, когда вы пересекаете границы между языками.

Сопоставление типов (Type Matching)

Типы не разделяются через границу FFI. Когда вы объявляете тип в Rust, информация об этом типе полностью теряется при компиляции. Другой стороне сообщаются только биты, составляющие этот тип. Поэтому вам нужно объявить тип для этих битов по обе стороны границы. Когда вы объявляете Rust-версию типа, вы сначала должны убедиться, что примитивы, содержащиеся в этом типе, совпадают. Например, если C используется по другую сторону границы, и C-тип использует `int`, то Rust-коду лучше использовать точный Rust-эквивалент: `i32`. Чтобы убрать часть догадок из этого процесса, для интерфейсов, использующих C-подобные типы, стандартная библиотека Rust предоставляет вам корректные C-типы в модуле `std::os::raw`, который определяет `type c_int = i32`, `type c_char = i8/u8` в зависимости от того, является ли `char` знаковым, `type c_long = i32/i64` в зависимости от ширины указателя целевой платформы и так далее.

ПРИМЕЧАНИЕ Обратите особое внимание на причудливые целочисленные типы в C, такие как `__be32`. Они часто не переводятся напрямую в Rust-типы, и их, возможно, лучше оставить как что-то вроде `[u8; 4]`. Например, `__be32` всегда кодируется как `big-endian`, тогда как `i32` в Rust использует порядок байтов текущей платформы.

С более сложными типами, такими как векторы и строки, обычно нужно делать сопоставление вручную. Например, поскольку C обычно представляет строку как последовательность байтов, оканчивающуюся нулевым байтом, а не как строку в кодировке UTF-8 с отдельно хранимой длиной, вы обычно не можете использовать строковые типы Rust через FFI. Вместо этого,

предполагая, что другая сторона использует C-стильное строковое представление, вам следует использовать типы `std::ffi::CStr` и `std::ffi::CString` для заимствованных и владеющих строк соответственно. Для векторов вам, скорее всего, понадобится использовать сырой указатель на первый элемент, а затем передавать длину отдельно — метод `Vec::into_raw_parts` может для этого пригодиться.

Для типов, которые содержат другие типы, такие как структуры и объединения (unions), вам также нужно иметь дело с вопросами компоновки и выравнивания. Как мы обсуждали в главе 3, Rust по умолчанию располагает типы неопределённым образом, так что вам как минимум нужно будет использовать `#[repr(C)]`, чтобы гарантировать, что у типа детерминированная компоновка и выравнивание, отражающие то, что (вероятно и хотелось бы надеяться) используется по ту сторону границы FFI. Если интерфейс также задаёт другие настройки для типа, такие как ручная установка выравнивания или удаление выравнивающих заполнителей (padding), вам нужно будет соответствующим образом скорректировать ваш `#[repr]`.

Rust-перечисление (enum) имеет несколько возможных C-стильных представлений в зависимости от того, содержит перечисление данные или нет. Рассмотрим перечисление без данных, такое как это:

```
enum Foo { Bar, Baz }
```

C `#[repr(C)]` тип `Foo` кодируется с использованием единственного целого числа того же размера, который C-компилятор выбрал бы для перечисления с тем же количеством вариантов. Первый вариант имеет значение 0, второй — значение 1 и так далее. Вы также можете вручную присвоить значения каждому варианту, как показано в листинге 12-3.

```
#[repr(C)]
enum Foo {
    Bar = 1,
    Baz = 2,
}
```

Листинг 12-3: Определение явных значений вариантов для перечисления без данных

ПРИМЕЧАНИЕ Технически спецификация говорит, что значение

первого варианта равно 0, а значение каждого последующего варианта на единицу больше предыдущего. Это имеет значение, если вы вручную задаёте значение для некоторых вариантов, но не для других, — те, которые вы не задаёте, продолжат отсчёт от последнего заданного.

Однако вам следует быть осторожными с подобным сопоставлением enum-подобных типов C с Rust, поскольку для экземпляра типа перечисления допустимы только значения определённых вариантов. Это часто приводит к проблемам с C-стильными перечислениями, которые нередко работают как наборы битов (bitsets), где варианты могут быть объединены побитовым ИЛИ для получения значения, инкапсулирующего сразу несколько вариантов. В примере из листинга 12-3, например, значение 3, полученное взятием `Bar | Baz`, не было бы допустимым для `Foo` в Rust! Если вам нужно смоделировать C-API, использующий перечисление для набора битовых флагов, которые можно устанавливать и сбрасывать по отдельности, рассмотрите использование newtype-обёртки вокруг целочисленного типа с ассоциированными константами для каждого варианта и реализациями различных трейтов `Bit*` для улучшения эргономики.

ПРИМЕЧАНИЕ Для перечислений без полей вы также можете передать числовой тип в `#[repr]`, чтобы использовать для дискриминанта другой тип, нежели `isize`. Например, `#[repr(u8)]` будет кодировать дискриминант с использованием одного беззнакового байта. Для перечисления, несущего данные, вы можете передать `#[repr(C, u8)]`, чтобы получить тот же эффект.

Для перечисления, которое содержит данные, атрибут `#[repr(C)]` заставляет перечисление быть представленным с помощью *размеченного объединения* (tagged union). То есть его представление в памяти — это представление `#[repr(C)]`-структуры с двумя полями, где первое является дискриминантом, как если бы оно было закодировано, если бы ни у одного из вариантов не было полей, а второе является объединением структур данных для каждого варианта. В качестве конкретного примера рассмотрим перечисление и связанное с ним представление в листинге 12-4.

```
#[repr(C)]
enum Foo {
    Bar(i32),
```

```

    Baz { a: bool, b: f64 }
}
// представлено как
#[repr(C)]
enum FooTag { Bar, Baz }
#[repr(C)]
struct FooBar(i32);
#[repr(C)]
struct FooBaz{ a: bool, b: f64 }
#[repr(C)]
union FooData {
    bar: FooBar,
    baz: FooBaz,
}
#[repr(C)]
struct Foo {
    tag: FooTag,
    data: FooData
}

```

Листинг 12-4: Rust-перечисления с `#[repr(C)]` представлены как размеченные объединения.

НИШЕВАЯ ОПТИМИЗАЦИЯ В FFI (THE NICHE OPTIMIZATION IN FFI) В главе 10 мы говорили о нишевой оптимизации (niche optimization), при которой Rust-компилятор использует недопустимые битовые паттерны для представления вариантов перечисления, не содержащих данных. Тот факт, что эта оптимизация гарантирована, приводит к интересному взаимодействию с FFI. В частности, это означает, что обнуляемые (nullable) указатели всегда можно представить в FFI-типах с помощью указательного типа, обёрнутого в `Option`. Например, обнуляемый указатель на функцию можно представить как `Option<extern fn(...)>`, а обнуляемый указатель на данные — как `Option<*mut T>`. Они прозрачно сделают правильную вещь, если будет предоставлено значение из всех нулевых битов, и представят его как `None` в Rust.

Аллокации (Allocations)

Когда вы выделяете память, эта аллокация принадлежит своему аллокатору и может быть освобождена только этим же аллокатором. Так обстоит дело, если вы используете несколько аллокаторов в Rust, а также если вы выделяете память и в Rust, и с помощью некоторого аллокатора по другую сторону границы FFI. Вы вольны передавать указатели через границу и обращаться к

этой памяти сколько душе угодно, но когда дело доходит до повторного освобождения памяти, она должна быть возвращена соответствующему аллокатору.

У большинства FFI-интерфейсов будет одна из двух конфигураций для обработки аллокации: либо вызывающая сторона предоставляет указатели на участки памяти, либо интерфейс предоставляет специальные методы освобождения, которым следует возвращать любые выделенные ресурсы, когда они больше не нужны. Листинг 12-5 показывает пример Rust-объявлений некоторых сигнатур из библиотеки OpenSSL, использующих память, управляемую реализацией.

```
// Одна функция выделяет память для нового объекта.  
extern fn ECDSA_SIG_new() -> *mut ECDSA_SIG;  
  
// А другая принимает такой указатель (созданный new)  
// и освобождает его, когда вызывающая сторона закончила с ним.  
extern fn ECDSA_SIG_free(sig: *mut ECDSA_SIG);
```

Листинг 12-5: Интерфейсы памяти, управляемой реализацией

Функции `ECDSA_SIG_new` и `ECDSA_SIG_free` образуют пару, где вызывающая сторона должна вызвать функцию `new`, использовать возвращённый указатель столько, сколько ей нужно (вероятно, передавая его, в свою очередь, другим функциям), а затем наконец передать указатель функции `free`, когда она закончила работу со ссылочным ресурсом. Реализация, предположительно, выделяет память в функции `new` и освобождает её в функции `free`. Если бы эти функции были определены в Rust, функция `new`, скорее всего, использовала бы `Box::new`, а функция `free` вызвала бы `Box::from_raw` и затем уничтожила бы значение, чтобы запустить его деструктор.

Листинг 12-6 показывает пример памяти, управляемой вызывающей стороной.

```
// Пример памяти, управляемой вызывающей стороной.  
// Вызывающая сторона предоставляет указатель на участок памяти,  
// который реализация затем использует для своих собственных типов.  
// Функция free не предоставляется, так как это происходит у вызывающей стороны.  
extern fn BIO_new_mem_buf(buf: *const c_void, len: c_int) -> *mut BIO
```

Листинг 12-6: Интерфейс памяти, управляемой вызывающей стороной

Здесь функция `BIO_new_mem_buf` вместо этого требует, чтобы вызывающая

сторона предоставила подкладочную память. Вызывающая сторона может выбрать выделить память в куче или использовать любой другой механизм, который она сочтёт подходящим для получения требуемой памяти, и затем передаёт её библиотеке. После этого обязанность вызывающей стороны — убедиться, что память позже освобождается, но только тогда, когда она больше не нужна FFI-реализации.

Вы можете использовать любой из этих подходов в своих FFI-API или даже смешивать их по своему усмотрению. В качестве общего эмпирического правила: позволяйте вызывающей стороне передавать память, когда это осуществимо, поскольку это даёт ей больше свободы управлять памятью так, как она сочтёт нужным. Например, вызывающая сторона может использовать высокоспециализированный аллокатор на некоторой кастомной операционной системе и не хотеть быть вынужденной использовать стандартный аллокатор, который использовала бы ваша реализация. Если вызывающая сторона может передавать память, она может даже вовсе избежать аллокаций, если вместо этого может использовать память на стеке или повторно использовать уже выделенную память. Однако имейте в виду, что эргономика интерфейса, управляемого вызывающей стороной, часто более запутанна, поскольку вызывающая сторона теперь должна делать всю работу по выяснению того, сколько памяти выделять, и настраивать это, прежде чем сможет вызвать вашу библиотеку.

В некоторых случаях вызывающей стороне может быть даже невозможно заранее знать, сколько памяти выделять, — например, если типы вашей библиотеки непрозрачны (и потому неизвестны вызывающей стороне) или могут меняться со временем, вызывающая сторона не сможет предсказать размер аллокации. Аналогично, если вашему коду приходится выделять больше памяти во время работы, например если вы строите граф на лету, объём необходимой памяти может динамически варьироваться во время выполнения. В таких случаях вам придётся использовать память, управляемую реализацией.

Когда вы вынуждены идти на компромисс, выбирайте память, выделяемую вызывающей стороной, для всего, что является либо *крупным*, либо *частым*. В таких случаях вызывающая сторона, скорее всего, более всего заботится о контроле над самими аллокациями. Для всего остального, вероятно, нормально, чтобы ваш код выделял память, а затем предоставлял функции-

деструкторы для каждого релевантного типа.

Колбэки (Callbacks)

Вы можете передавать указатели на функции через границу FFI и вызывать функцию по этим ссылкам, пока тип указателя на функцию имеет аннотацию `extern`, соответствующую соглашению о вызове функции. То есть вы можете определить `extern "C" fn(c_int) -> c_int` в Rust, а затем передать ссылку на эту функцию в C-код в качестве колбэка (callback), который C-код в конечном итоге вызовет.

Вам нужно быть осторожными при использовании колбэков, окружённых паниками (panics), поскольку раскрутка стека из-за паники за конец функции, которая является чем угодно, кроме `extern "Rust"`, является неопределённым поведением. Rust-компилятор в настоящее время автоматически аварийно завершит работу, если обнаружит такую панику, но это не всегда может быть тем поведением, которое вам нужно. Вместо этого вы можете захотеть использовать `std::panic::catch_unwind`, чтобы обнаружить панику в любой функции, помеченной `extern`, а затем перевести панику в ошибку, совместимую с FFI.

Безопасность (Safety)

Когда вы пишете Rust FFI-привязки (bindings), большая часть кода, который фактически взаимодействует с FFI, будет небезопасной (`unsafe`) и в основном будет вращаться вокруг сырых указателей. Однако ваша цель в конечном счёте — представить *безопасный* (safe) Rust-интерфейс поверх FFI. Для этого в основном нужно внимательно прочитать инварианты небезопасного интерфейса, который вы оборачиваете, и убедиться, что вы соблюдаете их во всём безопасном интерфейсе. Три наиболее важных элемента безопасной инкапсуляции внешнего интерфейса — это аккуратный захват `&` против `&mut`, корректная реализация `Send` и `Sync`, а также гарантия того, что указатели нельзя случайно перепутать. Далее я расскажу, как обеспечить каждый из них.

Ссылки и времена жизни (References and Lifetimes)

Если есть вероятность, что внешний код модифицирует данные по заданному указателю, убедитесь, что безопасный Rust-интерфейс имеет эксклюзивную

ссылку на соответствующие данные, взяв `&mut`. Иначе пользователь вашей безопасной обёртки может случайно прочитать из памяти, которую внешний код одновременно модифицирует, и наступит полный хаос!

Вы также захотите как следует использовать времена жизни (lifetimes) Rust, чтобы гарантировать, что все указатели живут столько, сколько требует FFI. Например, представьте внешний интерфейс, который позволяет вам создать `Context`, а затем позволяет создать `Device` из этого `Context` с требованием, чтобы `Context` оставался действительным столько, сколько живёт `Device`. В этом случае любая безопасная обёртка для интерфейса должна обеспечивать это требование в системе типов, заставляя `Device` хранить время жизни, ассоциированное с заимствованием `Context`, из которого был создан `Device`.

Send и Sync (Send and Sync)

Не реализуйте `Send` и `Sync` для типов из внешней библиотеки, если только эта библиотека явно не документирует, что эти типы потокобезопасны! Это работа безопасной Rust-обёртки — гарантировать, что безопасный Rust-код *не может* нарушить инварианты внешнего кода и тем самым вызвать неопределённое поведение.

Иногда вы можете даже захотеть ввести фиктивные (dummy) типы для обеспечения внешних инвариантов. Например, скажем, у вас есть библиотека цикла событий с интерфейсом, приведённым в листинге 12-6 (примечание: в оригинале — листинг 12-7).

```
extern fn start_main_loop();
extern fn next_event() -> *mut Event;
```

Листинг 12-7: Библиотека, которая ожидает однопоточного использования

Теперь предположим, что документация для внешней библиотеки утверждает, что `next_event` может вызываться только тем же потоком, который вызвал `start_main_loop`. Однако у нас нет типа, через который мы можем это обеспечить! Вместо этого мы можем воспользоваться приёмом из главы 4 и ввести дополнительное состояние-маркер для обеспечения инварианта, как показано в листинге 12-7 (примечание: в оригинале — листинг 12-8).

```
pub struct EventLoop(std::marker::PhantomData<*const ()>);
```

```
pub fn start() -> EventLoop {
    unsafe { ffi::start_main_loop() };
    EventLoop(std::marker::PhantomData)
}
impl EventLoop {
    pub fn next_event(&self) -> Option<Event> {
        let e = unsafe { ffi::next_event() };
        // ...
    }
}
```

Листинг 12-8: Обеспечение FFI-инварианта путём введения вспомогательных типов

Пустой тип `EventLoop` на самом деле никак не связан с лежащим в основе внешним интерфейсом, а скорее обеспечивает контракт, согласно которому вы вызываете `next_event` только после вызова `start_main_loop` и только в том же потоке. Часть «тот же поток» обеспечивается тем, что `EventLoop` не является ни `Send`, ни `Sync`, поскольку он хранит фантомный сырой указатель (который сам по себе не является ни `Send`, ни `Sync`).

Использование `PhantomData<*const ()>` для «отмены» автотрейтов `Send` и `Sync`, как мы делаем здесь, уродливо и косвенно. У Rust действительно есть нестабильная возможность компилятора, которая включает отрицательные реализации трейтов вроде `impl !Send for EventLoop {}`, но её на удивление сложно сделать правильно, и она вряд ли стабилизируется в ближайшее время.

Вы могли заметить, что ничто не мешает вызывающей стороне вызвать `start_main_loop` несколько раз, либо из того же потока, либо из другого потока. Как бы вы с этим справились, зависело бы от семантики рассматриваемой библиотеки, так что я оставляю это вам в качестве упражнения.

Путаница указателей (Pointer Confusion)

Во многих FFI-API вы не обязательно хотите, чтобы вызывающая сторона знала внутреннее представление каждого участка памяти, на который вы передаёте указатели. У типа может быть внутреннее состояние, в которое вызывающей стороне не следует лезть, или это состояние может быть трудно выразить кросс-языково-совместимым способом. Для таких ситуаций C-стильные API обычно предоставляют *указатели на void* (void pointers), записываемые как C-тип `void*`, который эквивалентен `*mut std::ffi::c_void` в

Rust. Указатель со стёртым типом, такой как этот, по сути *просто* указатель и не несёт никакой информации о том, на что он указывает. По этой причине такие виды указателей часто называют *непрозрачными* (opaque).

Непрозрачные указатели по сути играют роль модификаторов видимости для типов через границы FFI — поскольку сигнатура метода не говорит, на что указывается, у вызывающей стороны нет иного выбора, кроме как передавать указатель и использовать любые доступные FFI-методы, чтобы получить видимость в ссылочную память. К сожалению, поскольку один `*mut c_void` неотличим от другого, ничто не мешает пользователю взять непрозрачный указатель, возвращённый из одного FFI-метода, и передать его методу, который ожидает указатель на *другой* непрозрачный тип.

В Rust мы можем сделать лучше. Чтобы смягчить такого рода путаницу указательных типов, мы можем избежать использования `*mut c_void` напрямую для непрозрачных указателей в FFI, даже если фактический интерфейс требует `void*`, и вместо этого сконструировать разные пустые типы для каждого отдельного непрозрачного типа. Например, в листинге 12-8 (примечание: в оригинале — листинг 12-9) я конструирую два различных непрозрачных указателя, которые нельзя перепутать.

```
#[non_exhaustive] #[repr(transparent)] pub struct Foo(c_void);
#[non_exhaustive] #[repr(transparent)] pub struct Bar(c_void);
extern {
    pub fn foo() -> *mut Foo;
    pub fn take_foo(arg: *mut Foo);
    pub fn take_bar(arg: *mut Bar);
}
```

Листинг 12-9: Непрозрачные указательные типы, которые нельзя перепутать

Поскольку `Foo` и `Bar` оба являются типами нулевого размера, их можно использовать вместо `()` в сигнатурах `extern`-методов. Более того, поскольку теперь это различные типы, Rust не позволит вам использовать один там, где требуется другой, так что теперь невозможно вызвать `take_bar` с указателем, полученным обратно из `foo`. Добавление аннотации `#[non_exhaustive]` гарантирует, что типы `Foo` и `Bar` нельзя сконструировать вне этого крейта.

`bindgen` и скрипты сборки (`bindgen` and Build Scripts)

Составление карты Rust-типов и `extern`-объявлений для более крупной внешней библиотеки может быть довольно утомительным занятием. Крупные библиотеки обычно имеют достаточно большое количество сигнатур типов и методов, чтобы выписывание всех Rust-эквивалентов было трудоёмким. У них также достаточно крайних случаев и С-причуд, чтобы непременно нашлись паттерны, требующие более тщательного обдумывания при переводе.

К счастью, сообщество Rust разработало инструмент под названием `bindgen`, который значительно упрощает этот процесс, при условии что у вас есть С-заголовочные файлы для библиотеки, с которой вы хотите взаимодействовать. По сути, `bindgen` кодирует все правила и лучшие практики, которые мы обсуждали в этой главе, плюс ещё ряд других, и упаковывает их в настраиваемый генератор кода, который принимает С-заголовочные файлы и выдаёт соответствующие Rust-эквиваленты.

`bindgen` предоставляет отдельный исполняемый файл, который один раз генерирует Rust-код для С-заголовков, что удобно, когда вы хотите зафиксировать (check in) привязки в системе контроля версий. Этот процесс позволяет вам вручную доработать сгенерированные привязки, если это окажется необходимым. Если же, с другой стороны, вы хотите генерировать привязки автоматически при каждой сборке и просто включить С-заголовочные файлы в исходный код, `bindgen` также поставляется как библиотека, которую вы можете вызвать в кастомном *скрипте сборки* (build script) для вашего пакета.

ПРИМЕЧАНИЕ Если вы фиксируете привязки напрямую, имейте в виду, что они будут корректны только на той платформе, для которой были сгенерированы. Генерация привязок в скрипте сборки будет генерировать их специально для текущей целевой платформы, что с меньшей вероятностью вызовет несоответствия компоновки, связанные с платформой.

Вы объявляете скрипт сборки, добавляя `build = "<some-file.rs>"` в секцию `[package]` вашего `Cargo.toml`. Это говорит Cargo, что перед компиляцией вашего крейта он должен скомпилировать `<some-file.rs>` как отдельную Rust-программу и запустить её; только после этого он должен компилировать исходный код вашего крейта. У скрипта сборки также есть собственные

зависимости, которые вы объявляете в секции `[build-dependencies]` вашего `Cargo.toml`.

ПРИМЕЧАНИЕ Если вы назовёте свой скрипт сборки `build.rs`, вам не нужно объявлять его в `Cargo.toml`.

Скрипты сборки очень удобны при FFI — они могут скомпилировать поставляемую в комплекте C-библиотеку из исходников, динамически обнаружить и объявить дополнительные флаги сборки, передаваемые компилятору, объявить дополнительные файлы, которые Cargo должен проверять на изменения для целей перекомпиляции, и, как вы догадались, генерировать дополнительные исходные файлы на лету!

Хотя скрипты сборки очень гибки, остерегайтесь делать их слишком осведомлёнными о среде, в которой они выполняются. Хотя вы можете использовать скрипт сборки, чтобы определить, является ли версия Rust-компилятора простым числом или будет ли завтра дождь в Стамбуле, постановка вашей компиляции в зависимость от таких условий может привести к неожиданным сбоям сборки у других разработчиков, что ведёт к плохому опыту разработки.

Скрипт сборки может записывать файлы в специальный каталог, предоставляемый через переменную окружения `OUT_DIR`. Тот же каталог и переменная окружения также доступны Rust-исходникам во время компиляции, так что они могут подхватить файлы, сгенерированные скриптом сборки. Чтобы сгенерировать и использовать Rust-типы из C-заголовка, вы сначала заставляете ваш скрипт сборки использовать библиотечную версию `bindgen` для чтения файла `.h` и превращения его в файл, называемый, скажем, *bindings.rs*, внутри `OUT_DIR`. Затем вы добавляете следующую строку в любой Rust-файл вашего крейта, чтобы включить *bindings.rs* во время компиляции:

```
include!(concat!(env!("OUT_DIR"), "/bindings.rs"));
```

Поскольку код в *bindings.rs* автогенерируется, обычно лучше всего размещать привязки в их собственном крейте и давать ему то же имя, что и у библиотеки, для которой предназначены привязки, с суффиксом `-sys` (например, `openssl-sys`). Если вы не следуете этой практике, выпуск новых версий вашей библиотеки будет гораздо болезненнее, поскольку два крейта, которые компонируются с одной и той же библиотекой через ключ `links` в `Cargo.toml`, не

могут сосуществовать в одной сборке. По сути, вам пришлось бы заставить всю экосистему обновиться до новой мажорной версии вашей библиотеки сразу. Выделение лишь привязок в их собственный крейт позволяет вам выпускать новые мажорные версии крейта-обёртки, которые можно внедрять постепенно. Это разделение также позволяет вам выпустить ломающий (breaking) релиз крейта с этими привязками, если Rust-привязки меняются, — скажем, если сами заголовочные файлы обновляются или обновление `bindgen` приводит к небольшому изменению сгенерированного Rust-кода, — без необходимости *также* выпускать ломающий релиз крейта, который безопасно оборачивает FFI.

ПРИМЕЧАНИЕ Помните, что если вы включаете какие-либо типы из крейта `-sys` в публичный интерфейс вашего основного библиотечного крейта, изменение зависимости от крейта `-sys` на новую мажорную версию по-прежнему представляет собой ломающее изменение для вашей основной библиотеки!

Если ваш крейт вместо этого производит библиотечный файл, который вы намереваетесь дать использовать другим через FFI, вам также следует опубликовать C-заголовочный файл для его интерфейса, чтобы упростить генерацию нативных привязок к вашей библиотеке из других языков. Однако этот C-заголовочный файл нужно поддерживать в актуальном состоянии по мере изменения вашего крейта, что может стать обременительным по мере роста размера вашей библиотеки. К счастью, сообщество Rust также разработало инструмент для автоматизации этой задачи: `cbindgen`. Как и `bindgen`, `cbindgen` является инструментом сборки и тоже поставляется и как исполняемый файл, и как библиотека для использования в скриптах сборки. Вместо того чтобы принимать C-заголовочный файл и производить Rust, он принимает Rust и производит C-заголовочный файл. Поскольку C-заголовочный файл представляет основное машиночитаемое описание FFI вашего крейта, я рекомендую вручную просматривать его, чтобы убедиться, что автогенерированный C-код не слишком неуклюж, хотя в целом `cbindgen` обычно производит вполне разумный код. Если же нет — заведите баг!

C++ В этой главе я в основном сосредоточился на C, поскольку это язык, наиболее часто используемый для описания кросс-языковых интерфейсов для библиотек, с которыми можно компоноваться. Почти каждый язык программирования предоставляет какой-то способ

взаимодействия с C-библиотеками. Хотя C++ кажется очень близкородственным C, и многие известные библиотеки написаны на C++, это совсем другой зверь, когда дело доходит до FFI. Генерация типов и сигнатур для соответствия C-заголовку относительно проста, но это совсем не так для C++. На момент написания у `bindgen` есть приличная поддержка генерации привязок к C++, но им недостаёт эргономики. Например, вам обычно приходится самостоятельно вручную вызывать конструкторы, деструкторы, перегруженные операторы и тому подобное. Некоторые возможности C++, такие как специализация шаблонов, также вообще не поддерживаются. Если вам всё же приходится взаимодействовать с C++, я рекомендую попробовать крейт `sxx`.

Итоги (Summary)

В этой главе мы рассмотрели, как использовать ключевое слово `extern` для вызова из Rust во внешний код, а также как использовать его, чтобы сделать Rust-код доступным внешнему коду. Мы также обсудили, как согласовывать Rust-типы с типами по другую сторону границы FFI, и некоторые из распространённых ловушек при попытке заставить код, написанный на двух разных языках, хорошо сочетаться. Наконец, мы поговорили об инструментах `bindgen` и `cbindgen`, которые делают поддержание FFI-привязок в актуальном состоянии гораздо приятнее. В следующей главе мы рассмотрим, как использовать Rust в более ограниченных средах, таких как встраиваемые устройства, где стандартная библиотека может быть недоступна и где даже простая операция вроде выделения памяти может быть невозможна.